

backtesting, support for trading across multiple assets, integration with live trading brokers, and support for multiple programming languages. Lean Engine is appropriate for institutional-grade algorithmic trading research.

Vectorized vs event-driven backtesting

Backtesting frameworks are typically divided into two major architectures: event-driven backtesting and vectorized backtesting. Event-driven systems process market data in a sequential manner, comparable to real trading settings. They deliver realistic simulation and are appropriate for sophisticated strategies but are computationally expensive. Backtrader and Zipline are two tools available for this. Vectorized backtesting, on the other hand, is really fast since it uses matrix operations to evaluate strategies on huge datasets at the same time. Vectorize models are frequently used to optimise trade parameters for real-time trading.

VectorBT uses vectorization framework.

The challenges and best practices

Although backtesting is necessary, it has several drawbacks. It frequently suffers from 'look-ahead bias' due to the leakage of future information into the historical simulation, which results in inaccurate performance. Overfitting to historical data causes techniques to perform well on past information but fail in real-world scenarios. 'Data snooping' bias may result in an overly profitable situation solely by accident. Backtesting sometimes ignores trading costs in hypothetical trading, which can greatly overstate plan profitability and act as a bottleneck in strategy implementation.

To attain consistent results, traders should ensure that their techniques are resilient and general. So they must employ high-quality historical data, include actual transaction costs, eliminate look-ahead bias, execute out-of-sample testing, and conduct walk-forward validation.

Backtesting is evolving due to advances in computing and data science. Cloud-based backtesting environments, GPU-accelerated simulations, machine learning-driven strategies, and reinforcement learning trading agents are currently available for trading. These technologies have made it possible to evaluate complex models and large financial datasets more precisely and rapidly. **END** 🐧

By: Dr Dipankar Ray

The author is a researcher and tech enthusiast with interests in artificial intelligence, machine learning, natural language processing, and data-driven research. He is involved in academic projects, technical writing, and training programmes focusing on the practical applications of AI and computational techniques.

■ Continued from page...73

The road ahead

A few trends worth keeping an eye on. AI-powered API testing tools are getting genuinely useful, both for defenders and attackers. Tools like StackHawk and similar are starting to use LLMs to generate test cases that explore weird edge cases human testers would miss. Worth experimenting with.

API gateways are getting smarter. Modern ones can do anomaly detection, automatically block suspicious patterns, and integrate with identity providers in ways that would have been a major engineering effort a few years ago. If you're still rolling your own gateway logic, it might be worth seeing what's available off the shelf.

GraphQL is everywhere now and it brings its own set of security headaches. Query complexity attacks, where someone sends a deeply nested query that takes your server forever to resolve,

are common enough that you should specifically defend against them if you're using GraphQL. Depth limits and query cost analysis are your friends.

And the regulatory environment is tightening. Data protection laws around the world are becoming more demanding about how you handle API access, especially for personal data. Building security in from the start is going to be more cost-effective than retrofitting it when the rules change.

Code that doesn't trust anyone, including itself

The honest reality is that there's no point at which your API is "secure" and you can

stop thinking about it. Every new feature is a new surface. Every new integration is a new caller. Every new tool the attackers get is a new way in. The best you can do is build the habit of doubting your own code, validating every input, checking every permission, logging every meaningful action, and reviewing what you've shipped on a regular schedule. The teams that take this seriously rarely show up in the news. The ones that don't eventually do. Pick a few habits from this article, especially the ones that apply most to what you're building. Start this week. Your API doesn't need to be perfect. It just needs to be a less appealing target than the next one. Most of the time, that's enough. **END** 🐧

By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.